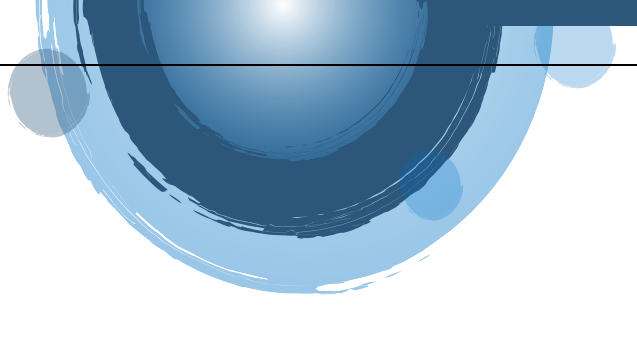




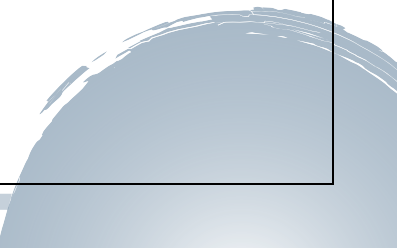
Cyber Security Internship Report

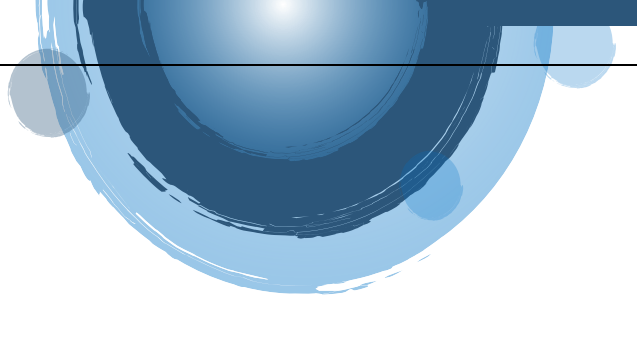
Cyber Security Internship – Task 13

Secure API Testing & Authorization Validation Report

Submitted by: Hetal Antala

Date : 06 / 02 / 2026



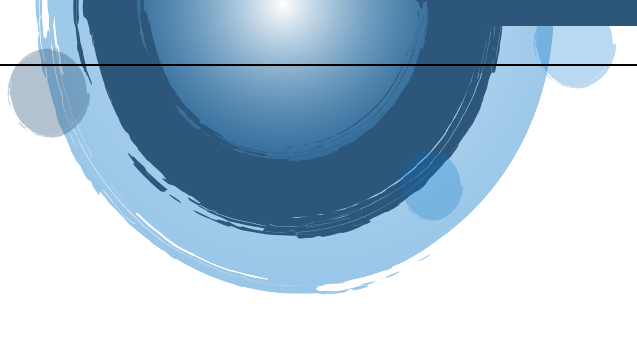


1. Introduction

Application Programming Interfaces (APIs) allow communication between frontend applications and backend servers using HTTP methods such as GET, POST, PUT, and DELETE. Modern web and mobile applications heavily rely on APIs for authentication, data exchange, and business logic.

Due to their exposure to the internet, APIs are common targets for cyberattacks. Improper authentication, broken authorization, injection flaws, and lack of rate limiting can lead to data breaches and unauthorized access.

This task focuses on performing security testing of REST APIs to identify common vulnerabilities and misconfigurations based on the OWASP API Security Top 10 risks.



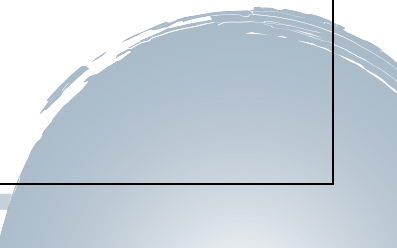
2. Objectives

The main objectives of this task are:

- Understand REST API architecture
 - Test authentication mechanisms
 - Validate authorization controls
 - Identify common API vulnerabilities
 - Perform manual and automated testing
 - Map findings to OWASP API risks
-

3. Tools Used

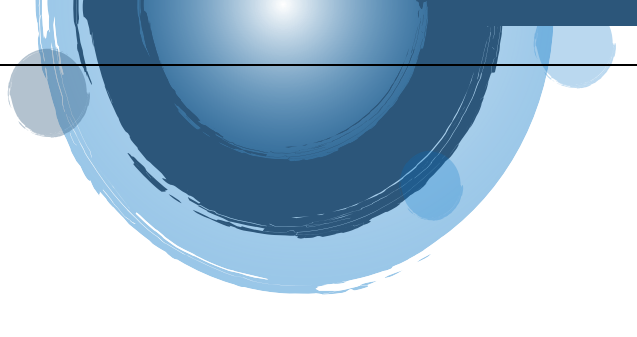
The following tools were used during testing:

- Postman – Manual API request testing
 - curl – Command-line API testing
- 

- OWASP Juice Shop – Vulnerable test application
 - Docker – Running Juice Shop locally
 - Browser Developer Tools – Traffic inspection
-

4. Test Environment

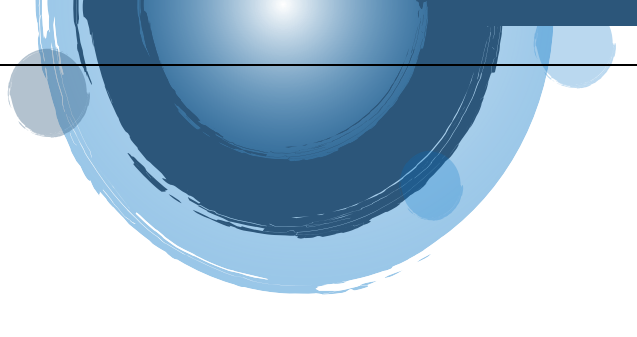
Component	Details
-----	-----
OS	Kali Linux
Target App	OWASP Juice Shop
Deployment	Docker container
Port	3000
API Base URL	http://localhost:3000/rest



5. Methodology

The testing process followed a structured approach:

1. Deploy vulnerable API locally using Docker
 2. Identify API endpoints
 3. Send baseline requests
 4. Test authentication controls
 5. Test authorization (IDOR)
 6. Send malformed/malicious inputs
 7. Perform rate limiting tests
 8. Analyze HTTP responses and error handling
 9. Document vulnerabilities
 10. Capture screenshots as evidence
-



6. Tests Performed

Test 1 – Normal API Request (Baseline)

Purpose

To verify that the API is reachable and functioning correctly.

Command

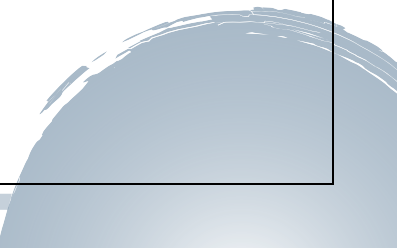
```
curl -H "Accept: application/json"  
http://localhost:3000/rest/products
```

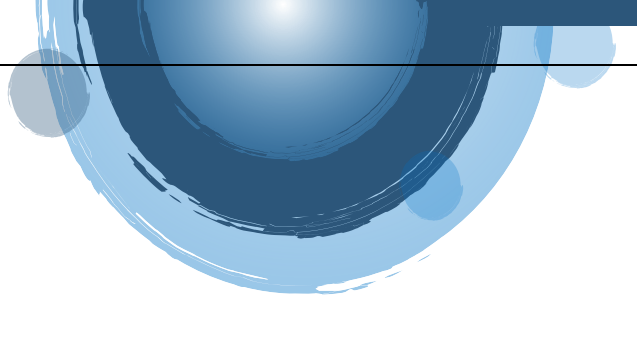
Observation

- Status Code: 200 OK
- Returned product list in JSON

Result

Application working normally.





Test 2 – Authentication Validation

Purpose

Check whether protected endpoints require authentication.

Command

```
curl http://localhost:3000/rest/user/whoami
```

Observation

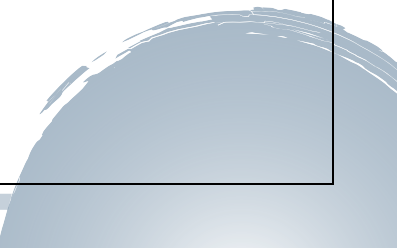
- Status Code: 401 Unauthorized

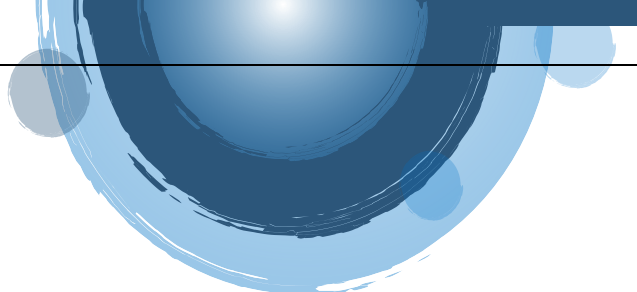
Result

Authentication properly enforced.

Test 3 – Login and Token Generation

Purpose





Verify login functionality and token-based authentication.

Command

POST /rest/user/login

Observation

- JWT token generated successfully

Result

Authentication mechanism working.

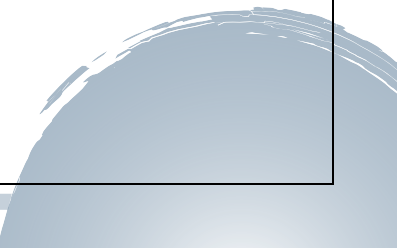
Test 4 – Broken Authorization (IDOR)

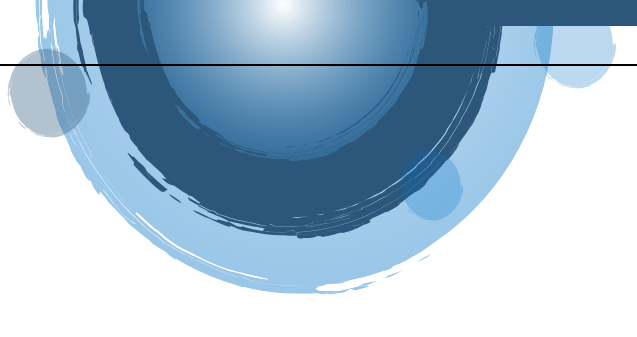
Purpose

Test if users can access other users' resources.

Command

/rest/basket/1





/rest/basket/2

/rest/basket/3

Observation

- Able to view other users' baskets

Risk

Unauthorized data exposure

Result

Broken Object Level Authorization detected

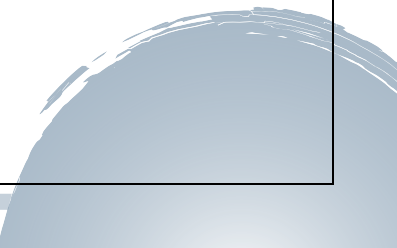
Test 5 – Injection Testing (SQL Injection)

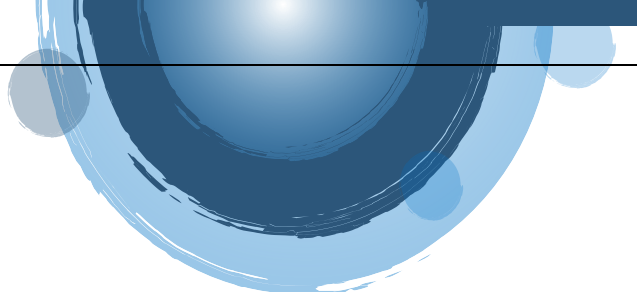
Purpose

Check for improper input validation.

Payload

' OR 1=1 --





Observation

- Login bypass successful

Risk

Database compromise

Result

SQL Injection vulnerability present

Test 6 – Malicious Input (XSS)

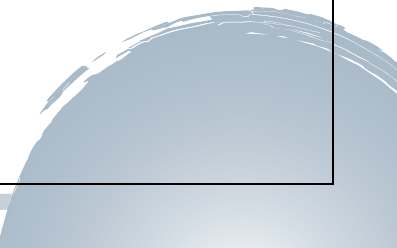
Purpose

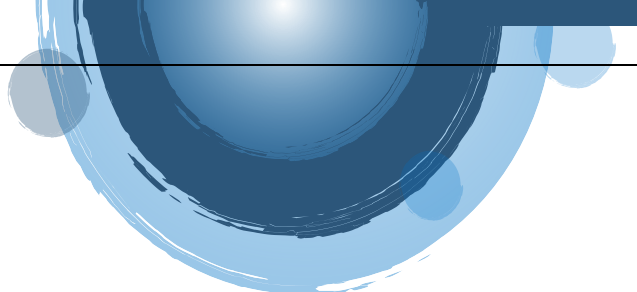
Check if scripts are sanitized.

Payload

```
<script>alert(1)</script>
```

Observation

- Input stored without sanitization
- 



Risk

Cross-Site Scripting (XSS)

Result

Input validation weakness

Test 7 – Rate Limiting

Purpose

Check protection against brute force or DoS attacks.

Command

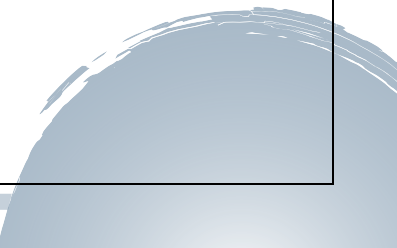
Multiple rapid requests using loop

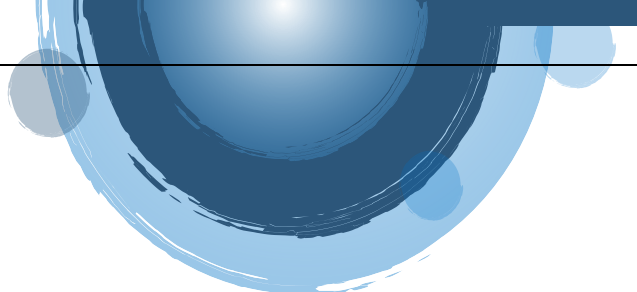
Observation

- No blocking or throttling

Risk

Server overload or abuse





Result

No rate limiting implemented

Test 8 – Error Handling / Information Disclosure

Purpose

Check if internal errors are leaked.

Command

/rest/invalidendpoint

Observation

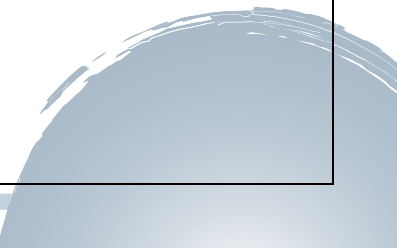
- Detailed stack traces displayed

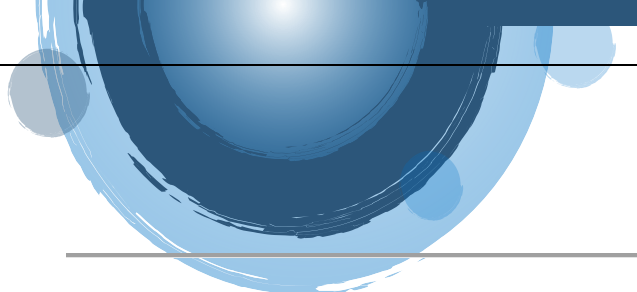
Risk

Information leakage

Result

Improper error handling

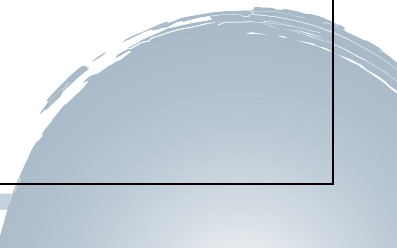


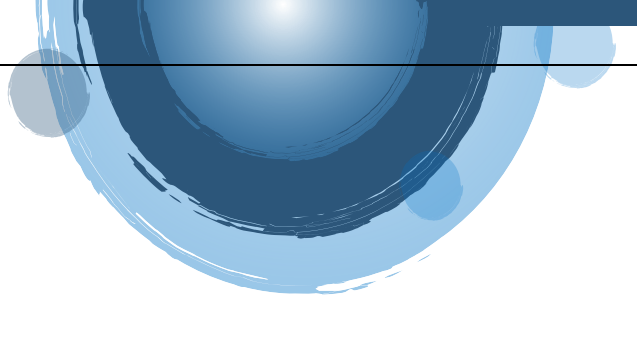


7. Vulnerabilities Identified

#	Vulnerability	Description	OWASP API Risk
1	Broken Authorization (IDOR)	Access other users' data	API1
2	SQL Injection	Login bypass possible	API8
3	Stored XSS	Malicious scripts accepted	API8
4	No Rate Limiting	Unlimited requests allowed	API4
5	Information Disclosure	Detailed errors shown	API7

8. OWASP API Risk Mapping

- API1 – Broken Object Level Authorization
 - API2 – Broken Authentication
- 

- 
- API4 – Lack of Rate Limiting
 - API7 – Security Misconfiguration
 - API8 – Injection
-

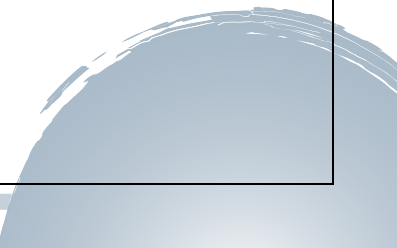
9. Recommendations

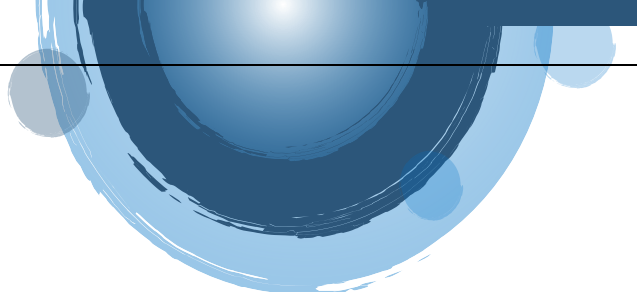
Implement proper authorization checks

- Use parameterized queries
 - Validate and sanitize inputs
 - Add rate limiting
 - Hide detailed server errors
 - Use strong authentication tokens
 - Enable logging and monitoring
-

10. Conclusion

API security is critical for protecting user data and application integrity. Improper implementation of





authentication and authorization can lead to serious breaches.

The performed testing successfully identified multiple vulnerabilities, demonstrating the importance of secure coding practices and regular security assessments. This exercise enhanced my understanding of API penetration testing methodologies and tools.
